

Secure Programming – C Vulnerabilities and Overflow Issues |

Madison Roche – Student Number: 20112525

Introduction.....	2
1.1 Source Code	4
1.2 Vulnerable Code Overview	4
1.3 Compilation and Execution	5
1.3.1 Vulnerable Version	5
1.3.2 Fixed Version.....	6
1.4 Explanation of Overflow	7
1.5 Secure Fix and Resolution	8
Example 2 – String Format Overflow.....	8
2.1 Source Code	9
2.2 Vulnerable Code Overview	9
2.3 Compilation and Execution	9
2.3.1 Vulnerable Version	10
2.3.2 Fixed Version.....	11
2.4 Explanation of Overflow	12
2.5 Secure Fix and Resolution	13
Example 3 – Integer Overflow	14
3.1 Source Code	15
3.2 Vulnerable Code Overview	15
3.3 Compilation and Execution	16
3.3.1 Vulnerable Version	16
3.3.2 Fixed Version.....	17
3.4 Explanation of Overflow	18
3.5 Secure Fix and Resolution	20
General Prevention of Overflow Vulnerabilities	20
Conclusion.....	21
Bibliography	21

Introduction

An overflow error occurs when a computer attempts to store data within a fixed-size area of storage but is inefficient due to it being fixed-size. This can happen due to the lack of needed validation of inputs, wrong numbers due to calculation errors, and increased the size of a number as it moves through the code. (Fiveable, 2025)

When overflows occur, they can lead to several different errors and highly unpredictable behaviour within the program. They can lead to crashes, incorrectly produced results, and sometimes, the compromise of security and stability of the system. Data can also be overwritten, and code can even be executed because of a lack of positive programming practices. (Lenovo, 2025)

There are several diverse types of overflow vulnerabilities specifically in C, but the ones I am focusing on in this assignment are as follows:

1. Stack Buffer Overflows
2. Format String Overflows
3. Integer Overflows

Each of these overflows occur from the lack of proper validation, unsafe usage of functions, and overall, just poor programming practices. They all produce slightly different results with different theories and ways of occurring. These overflows are the ones that I have decided to recreate and analyse in my investigation into C vulnerabilities and overflow issues.

Each following example includes a further investigation into the three overflows, creating the code and demonstrating how to fix it to avoid overflow issues. There are demonstrations and some mathematics of how the code works, how the overflow happens, and the potential results it can yield if exploited in the field. The examples I have coded will become more challenging and detailed as the report progresses as I wanted to challenge myself and to learn more about coding in C that I already knew.

All the examples within the code were coded by me and not pulled from other sources. In the examples, I am compiling them using GCC on the command line, but within the GitHub repository there are shell scripts to compile them in the same manner.

It is important to learn about overflow vulnerabilities as it helps avoid easily made mistakes in the past. Historically, overflow vulnerabilities have often been exploited in major cybersecurity attacks, demonstrating their importance to understand and protect against them in all areas of secure programming and scripting practices.

Example 1 – Stack Buffer Overflow

1.1 Source Code

This first example is a simple program that causes a stack buffer overflow. This demo involves a user inputting their name through a command line input. The program stores this input in a fixed-size array and maintains a constant integer value that represents whether the user has administrative access or not.

The integer **isAdmin** variable, representing the user has access of that level, by default, **is set to be a value of 0**, meaning no matter what the user inputs it will always deny administrator access. The name inputted by the user is stored in a character array called **name**. The **name** buffer is limited to a size of 8 characters, `char name[8]`. The user's input is read by the **scanf()** function and stores it into the buffer.

The code then checks the value of **isAdmin**. If **isAdmin** is less than 1, it will print to the command line that the user does not have admin privileges and will print out the number of the integer that has been checked. By default, if no overflow occurs, this scenario should always happen, and the number read back out should be 0. If the overflow occurs, it should print out that the user does have administrative access and will print out the integer stored in **isAdmin** that has caused the error.

1.2 Vulnerable Code Overview

The first issue with this code is the fixed-size character array that is acting as the buffer that the command line input is read into by `scanf()`. The character array **name** is declared as an array that can hold eight characters. The null terminator is also included in this count of eight, which means that at most, the only safe string that can be inputted by the user is either less than or equal to seven.

The program uses the line of code `scanf("%s", name);` to read in the inputted name. The formatter, `%s`, tells the code to continue reading until it finds a whitespace. As there is no restriction coded into the formatter, it will keep on reading until it hits the whitespace, not the cap on the character array.

Stack memory stores local variables in contiguous memory. In the program, the **name** buffer and the **isAdmin** integer are both placed on the stack after one another. When a name longer than the buffers capacity is entered, it will continue into the extra bytes beyond the distributed space. These extra bytes write into the memory directly beside the **name** buffer, altering the data stored for **isAdmin** which changes the number that is held there to a non-zero integer. (Kartik, 2025)

1.3 Compilation and Execution

1.3.1 Vulnerable Version

The program is compiled and ran using the following:

```
$ gcc -o stackOverflow.o stackOverflow.c -fno-stack-protector
$ ./stackOverflow.o
```

Figure 1.0 - Compiling and running stackOverflow.o

When the program is run with a short user input, it behaves as expected by the user. It will reprint your entered name and then check to see what your admin level is set to. By default, it is set to zero and will always stay at zero as long as less than seven characters are entered. If the inputted name is less than seven characters, there will be no issues with the program, and it will tell you that you do not have admin privileges.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: maddy
Hello maddy
You do not have Admin Privilages (0)
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: madison
Hello madison
You do not have Admin Privilages (0)
```

Figure 1.1 - Inputting a string/name less than or equal to seven characters.

To force the overflow to occur, an inputted string of 13 characters or more must be entered into the input for name. The maths about this is reasoned in the next section as this confused me at the beginning before I done an investigation into why it acted that way. It is important to see the difference between a 12-character input and a 13-character input.

```

maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: madisonnnnnn I
Hello madisonnnnnn
You do not have Admin Privilages (0)
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: madisonnnnnnn
Hello madisonnnnnnn
You have Admin Privilages (110)
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ █

```

Figure 1.2 - 12-character input vs. 13-character input

When the 12-character input is used, the admin privileges level stays set to zero. When the 13-character input is used, the overflow finally occurs, which is signalled by the changed admin privileges number which is now set to 110 rather than 0. The reason the number for isAdmin is altered is because the extra characters overwrite the memory locations used by isAdmin, altering the value stored in that variable.

1.3.2 Fixed Version

The program is compiled and ran using the following:

```

$ gcc -o stackOverflowFixed.o stackOverflowFixed.c -fno-stack-protector
$ ./stackOverflowFixed.o

```

Figure 1.3 - Compiling and running stackOverflowFixed.o

I have decided to compile it with the argument `-fno-stack-protector` as it will allow the fixes to become obvious rather than the program protecting against it itself.

When the program is run it is visually similar to the first and with a short user input, it behaves as expected by the user. It will reprint your entered name and then check to see what your admin level is set to. By default, it is set to zero and will continue to remain as zero as this fixed version has protections against the overflow.

```

maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./stackOverflowFixed.o
Enter your name: maddy
Hello maddy
You do not have Admin Privilages (0)
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./stackOverflowFixed.o
Enter your name: madison
Hello madison
You do not have Admin Privilages (0)

```

Figure 1.4 - Inputting a string/name less than seven characters

In the vulnerable version, the overflow occurred when the input exceeded more than 12 characters which began to read into the memory of **isAdmin** and altering the value. When we attempt to do this again, an error message is printed to the command line which tells the reader that the string is too long and instantly stops running the program.

```

maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./stackOverflowFixed.o
Enter your name: madisonnnnnn
Error: name is too long!
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./stackOverflowFixed.o
Enter your name: madisonnnnnnnnnnn
Error: name is too long!
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./stackOverflowFixed.o
Enter your name: madisonn
Error: name is too long!

```

Figure 1.5 - Inputting a string/name longer than seven characters

The fixed program has multiple defences against any possible overflow issues and attacks, those of which are explained in Section 1.5.

1.4 Explanation of Overflow

One question I had been constantly asking myself during the testing of the vulnerable program, was ‘why does it only overflow when 13 characters are inputted, even when the array is set to hold 7?’

To better understand the behaviour of the overflow, I decided to investigate the variables places in memory. This would allow me to see whether the variables had some sort of padding between them which pushed the character limit to trigger a visible overflow, or if it was the fault of something else.

I first printed the starting address of name and the starting address of **isAdmin** using a simple, temporary, two lines to my code. From the screenshot below, I will choose the first set of addresses. The two addresses are:

- Address of **name**: 0xffffcbf75ba0
- Address of **isAdmin**: 0xffffcbf75bac

```

maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: madisonnnnnn
Hello madisonnnnnn
You do not have Admin Privilages (0)
Address of name: 0xffffcbf75ba0
Address of isAdmin: 0xffffcbf75bac
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./stackOverflow.o
Enter your name: madisonnnnnnn
Hello madisonnnnnnn
You have Admin Privilages (110)
Address of name: 0xffffc510e6a0
Address of isAdmin: 0xffffc510e6ac
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ █

```

Figure 1.6 - Investigating the addresses of both name and isAdmin variables

The difference between the two addresses is found to be a total of 12 bytes. As we know, the name buffer is only set to occupy 8 of these 12 bytes, meaning that there has been a padding of 4 bytes inserted between the end of **name** and the beginning of **isAdmin**. We can confirm it is 12 bytes between if we run the program again and recount the number of bytes between, which in my investigation does stay consistent.

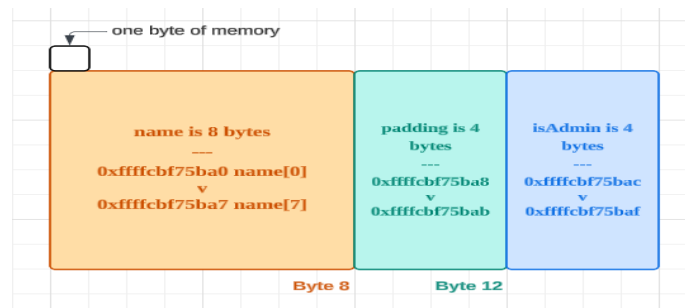


Figure 1.7 - Visual layout of the reserved memory for name, padding, and isAdmin

This lets us know that when the 13 character is written into memory, it begins to occupy the starting address of **isAdmin** as it is officially past the padding and into the memory that **isAdmin** begins in.

1.5 Secure Fix and Resolution

The vulnerability in this demo can be easily resolved by guaranteeing that the user input does not exceed the size of the buffer. One way to do this is to limit the number of characters that `scanf("%s", name);` reads to name. For example, in my code that line I have changed to `scanf("%7s", name);` to account for the limited seven characters and the null terminator of the string. By constraining the input length this way, the program can prevent data from reading over the distributed amount and cut it off before it causes issues.

In addition, an extra validation step was added to detect whether the user tried to input too many characters. After reading the name, the program then tries to read the next character from the stream into the variable **extra**. If one added character is read and it is not a newline character, the program will print an error message and at once stop running before the overflow can occur.

This two-step validation to protect against the overflow vulnerability is robust by both preventing the overflow and warning the user that the input exceeds the allowed length. By having these cautions and checks in place, it removes the possibility of overwriting **isAdmin** and causing the overflow in the first place.

Example 2 – String Format Overflow

2.1 Source Code

This example is based around a string format vulnerability that simulates a password input situation. The program asks the user to enter a password and compares the entered password to a defined password using **strcmp()** function. The password is checked using `(strcmp(password, "secret123") == 0)`. If the result is true and the strings are equal (0), the variable **authenticated** is updated to be one. If the result is false and there is a difference between the strings, it will reprint the incorrect password and tell you if it is wrong.

In the vulnerable code, the code reprints the inputted password using `printf(password);`. Within format string overflows, there is a lack of a format specifier within the code, the input string being treated as the format string itself or as a command. This allows attackers to use format specifiers such as **%x** or **%s** to cause the program to read different values from memory. It also shows the substantial difference between **%s** and **“%s”**, one acting as the format string and the other acting as a string. (Fortinet, n.d.)

There is also opportunity for a stack overflow to occur here as there is no limitations in place for when the password is scanned in using `scanf(“%s”, password)` but the buffer for the password is **char password[10]**.

2.2 Vulnerable Code Overview

As mentioned above, the vulnerability stems from the user-controlled input that is passed into the **printf()** function without specifying a format string. When the format string is controlled by the user and there is a lack of protection against it, the user can insert different format specifiers.

Depending on the format of specifiers inputted by the user, different types of information can be recovered from the stack and read. Different types of information that can be recovered could be anything from strings of text to memory values from the stack instead of the intended incorrectly entered password. (him0000, 2025)

In this program, the vulnerability appears when the user enters a format specifier into the password input rather than a string of text. This has unusual consequences and will allow for sensitive information to be read back to the user.

2.3 Compilation and Execution

2.3.1 Vulnerable Version

The program is compiled and ran using the following:

```
$ gcc -w -o formatStringOverflow.o formatStringOverflow.c -fno-stack-protector
$ ./formatStringOverflow.o
```

Figure 2.0 - Compiling and running stringFormatOverflow.o

I have decided to compile it with the argument `-fno-stack-protector` as it will allow the fixes to become obvious rather than the program protecting against it itself. It is also compiled with the `-w` argument as this tells GCC to leave out any warnings as you compile.

When the program is run it will ask for you to input a password. The password that the program is set to expect is 'secret123'. If the input is anything other than that password, it will return the incorrect password and print that the incorrect password was entered. If the correct password is entered, it will print out that you've gained access but won't reprint the password. The password is only reprinted if it is the incorrect password.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./formatStringOverflow.o
Enter password: password123
password123
Wrong password!
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./formatStringOverflow.o
Enter password: secret123
Access granted.
```

Figure 2.1 - Inputted an incorrect and then the correct password

As explained above, the overflow occurs because the input into the password scanner is not formatted, which allows us to enter in format specifiers to pull information from the program. In my example, the two format specifiers I have experimented with are `%x`, `%s`, and `%p`.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./formatStringOverflow.o
Enter password: %x%x%x
32f809204acae86264
Wrong password!
```

Figure 2.2 - Entering %x to see what is returned to the command line

The first is the formatter for hexadecimal integers. The second is the formatter for a string of characters, allowing to have strings read back to the command line. The third is the formatter for addresses, allowing us to view where in memory this information is being sent to and stored. By using `%s`, the proper password revealed to the user!

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./formatStringOverflow.o
Enter password: %s
secret123
Wrong password!
```

Figure 2.3 - Entering %s to see the correct password returned to the command line

The correct password is printed to the command line because there is a lack of a format specifier where the incorrect password is replaced and printed. The password appears due to `printf()` reading random values from the stack to fill the space and one of these stack locations holds a pointer to the string 'secret123' that exists in the program's memory.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./formatStringOverflow.o
Enter password: %p
0xc6e0bd780920
Wrong password!
```

Figure 2.4 - Entering %p to see the memory address that the password is stored at

The output given by the `%p` format specifier reveals memory addresses from the program's stack. As the user input is treated as the format string, `printf()` reads a value from the stack and converts it to a hexadecimal pointer to print. Attackers can use these printed addresses to help them understand the layout of a program's memory.

These examples show how format string vulnerabilities can expose a range of different sensitive information and internal memory that is usually hidden from the user.

2.3.2 Fixed Version

The program is compiled and ran using the following:

```
$ gcc -o formatStringOverflowFixed.o formatStringOverflowFixed.c -fno-stack-protector
$ ./formatStringOverflowFixed.o
```

Figure 2.5 - Compiling and running formatStringOverflowFixed.o

The fixed version is compiled the same way as the vulnerable version, leaving out the `-w` argument as no warnings arises from this code.

When the program is run it will ask for you to input a password. The password that the program is set to expect is 'secret123'. If the input is anything other than that password, it will return the incorrect password and print that the incorrect password was entered. If the correct password is entered, it will print out that you've gained access but won't reprint the password. The password is only reprinted if it is the incorrect password, like the vulnerable version.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./formatStringOverflowFixed.o
Enter password: password123
password123
Wrong password!
```

Figure 2.6 - Testing with incorrect password

When you now input the format specified that originally caused the code to malfunction and created the stack overflow, they are returned as strings in the incorrect passwords section. They aren't counted as the formatter for password anymore, but rather the string input for password itself. This means that no data on the stack is accessible anymore and that causing information leaks is not possible anymore.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./formatStringOverflowFixed.o
Enter password: %s
%s
Wrong password!
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./formatStringOverflowFixed.o
Enter password: %x
%x
Wrong password!
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./formatStringOverflowFixed.o
Enter password: %p
%p
Wrong password!
```

Figure 2.7 - Testing the format specifiers to see the changes made to the vulnerable code.

Finally, when the correct password is inputted, the program works the same way that the vulnerable version does, confirming that the correct password was entered and that access is granted.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./formatStringOverflowFixed.o
Enter password: secret123
Access granted.
```

Figure 2.8 - Testing the correct password

2.4 Explanation of Overflow

This format string overflow issue is quite easy to understand. In C, the **printf()** function expects the first argument that is entered to be the format string which tells the function how to interpret the following arguments. (*Fprintf Documentation*, 1997)

The lack of format specifier leaves the opportunity for the user inputted string to be classified at the format specifier itself. If any other information is passed into the **printf()** function while there is a lack of specification, this information will be affected by the format specifier the user inputted. Inputs from the user like `%x`, `%s`, and `%p` can both break

and reveal sensitive information that the application uses. This can allow the attacker to access the stack or buffers to see sensitive information.

This demonstration shows how format string vulnerabilities are easy to exploit and can allow attackers to read internal program memory, hidden strings, and view sensitive data.

2.5 Secure Fix and Resolution

The easiest way to make the program secure is to ensure that the user input is always treated as data rather than as an inputted format string. The overflow occurs when the incorrect password is read back to the user after being entered, and it is obvious from the code that this is where the necessary formatting is lacking as the password is read back from the stack.

I changed the line of code reading `printf(password);` to `printf("%s", password);` to ensure that whatever is entered into the input is guaranteed to be considered as a string and nothing else. This changes an input of `%s` to `"%s"`, ensuring that if `%s` is entered, a string reading `"%s"` is returned and won't leak any information.

Additionally, when reading the password in with `scanf("%s", password)`, there is no limit on the number of characters that can be entered. Although the character buffer **password** is set to hold 100 characters, a limitation of 99 characters should be implemented to ensure that a stack overflow doesn't occur also. The reflected and fixed code is `scanf("%99s", password)`, which ensures that the input cannot reach over 99 characters and still allows space for the null terminator at the end of the string, making the total count of characters 100.

Example 3 – Integer Overflow

3.1 Source Code

This example is based around an integer overflow vulnerability using a payment processing scenario. The program simulates a transaction or payment into a bank account, the account having a current balance and a limit on the amount that can be stored in the account at a time. These two numbers are set to a default, balance = 900 and limit = 1000. The user is then asked to input a number, a payment amount, which is then added to the existing balance if it is between 0 and 100.

All the variables that are to do with numbers are stored as **unsigned short**. I chose this data type for the vulnerability as it has a limited capacity, only holding values ranging from **0 to 65,535** as it has a maximum of 16 bits. It also only has all non-zero values. The program takes the inputted payment amount and calculates a new balance by adding the payment to the existing balance, then checking to see if it is within the allowed limit or not.

If it is within the limit, the payment is allowed to go through. If it isn't within the limit, the program rejects the transaction and stops. The goal of this example is to showcase how integer overflows occur through arithmetic operations and how the wrapping of numbers happens when they are too large to store. When the number wraps, it can cause a lot of errors and even increase, decrease, or become negative or positive.

3.2 Vulnerable Code Overview

The vulnerability originates from the arithmetic calculations that take place within the program. The result is not verified to see if it exceeds the amount that the data type can store. In the vulnerable version of the code, the program calculates **newBalance** using the expression:

```
unsigned short newBalance = balance + payment;
```

I have used the unsigned short data type to store the variables; the arithmetic calculations are limited to work between **0 and 65,535**. If the sum of balance and payment is more than **65,535**, the result is unable to be represented within the number of allocated spaces that is has available which is 16 bits. (Microsoft Learn, 2024)

When this occurs, the value attempts to wrap around due to modular arithmetic functions that the system does to allow the code to still be stored. Instead of producing an absurdly large value, the result resets to a smaller number within the range so it can be stored and properly represented. The wrapped value is then used in the program logic when checking whether the balance exceeds the allowed limit.

This will, depending on the range of values, allow for large payments to cause the calculated balance to be smaller than the limit, allowing the program to accept the transition as it deems it safe and within limits.

3.3 Compilation and Execution

3.3.1 Vulnerable Version

The program is compiled and ran using the following:

```
$ gcc -o integerOverflow.o integerOverflow.c -fno-stack-protector  
$ ./integerOverflow.o
```

Figure 3.0 - Compiling and running integerOverflow.o

I have decided to compile it with the argument `-fno-stack-protector` as it will allow the fixes to become obvious rather than the program or operating system protecting against it itself.

When the program is run it will ask for you to input a payment amount. The program also states that there's a current balance of 900 and a limit of 1000 on the account. If the input causes the newBalance to be more than the limit, it will return an error message that the payment was rejected and stop. If an amount for the payment is entered and the newBalance is calculated to be less than the limit, it will print out that the payment was accepted and print the newBalance to the command line. The newBalance is only reprinted if the payment amount is accepted.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./integerOverflow.o
Balance = 900, Limit = 1000
Enter Payment:
101
Payment Rejected...
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./integerOverflow.o
Balance = 900, Limit = 1000
Enter Payment:
97
Payment Accepted, New Balance = 997
```

Figure 3.1 - Inputting rejected and accepted amounts for payment

As we know, the unsigned short data type can only hold a maximum space of 65,535. If we try this number for the payments, we can see that the new balance is returned as 899, even though it should equal a higher number than that or should've been rejected as it would've resulted in a number above the limit. This is an example of where the overflow occurs.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./integerOverflow.o
Balance = 900, Limit = 1000
Enter Payment:
65535
Payment Accepted, New Balance = 899
```

Figure 3.2 - Entering 65,535 for the payment

If we try another random number, for example 70,000, we can see how the program reacts to it and stops running as both 70,000 and its numerically wrapped version both exceed the given buffer.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/vulnerable/src$ ./integerOverflow.o
Balance = 900, Limit = 1000
Enter Payment:
70000
Payment Rejected...
```

Figure 3.2 - Entering 70,000 for the payment

3.3.2 Fixed Version

The program is compiled and ran using the following:

```
c$ gcc -o integerOverflowFixed.o integerOverflowFixed.c -fno-stack-protector
c$ ./integerOverflowFixed.o
```

Figure 3.5 - Compiling and running integerOverflowFixed.o

I have decided to compile it with the argument `-fno-stack-protector` as it will allow the fixes to become obvious rather than the program or operating system protecting against it itself.

When the program is run it will run in the exact same way as the first. It will ask for you to input a payment amount. The program also states that there's a current balance of 900 and a limit of 1000 on the account. If the input causes the newBalance to be more than the limit, it will return an error message that the payment was rejected and stop. If an amount for the payment is entered and the newBalance is calculated to be less than the limit, it will print out that the payment was accepted and print the newBalance to the command line. The newBalance is only reprinted if they payment amount is accepted.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./integerOverflowFixed.o
Balance = 900, Limit = 1000
Enter Payment:
101
Payment Rejected...
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./integerOverflowFixed.o
Balance = 900, Limit = 1000
Enter Payment:
97
Payment Accepted, New Balance = 997
```

Figure 3.6 - Inputting rejected and accepted amounts for payment

In the vulnerable code when we input a number that is larger than or equal to 65,535, the code wraps around and causes the conversion overflow to happen. To see the difference between how the program accepts it now, we can test it by inputting the same number.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./integerOverflowFixed.o
Balance = 900, Limit = 1000
Enter Payment:
65535
Payment Rejected...
```

Figure 3.7 - Entering 65,535 for the payment

Testing 70,000 should still show us the same result as the vulnerable code as it is still above the limit regardless of the order of it being wrapped and stored in memory and the arithmetic functions that calculate the newBalance.

```
maddy@maddyubuntu:~/ca2-c-vulnerabilities-madison1106/fixed/src$ ./integerOverflowFixed.o
Balance = 900, Limit = 1000
Enter Payment:
70000
Payment Rejected...
```

Figure 3.8 - Entering 70,000 for the payment

3.4 Explanation of Overflow

In this example, there are two different overflows occurring depending on the amount that the payment is inputted to be. One is an arithmetic overflow caused during the calculation, the other is an integer wraparound during the input section overflow during the payment input stage.

The input conversion overflow begins at the very beginning when the user is prompted to enter a number for the payment. An unsigned short, which is the data type used in this example, can hold to a maximum of 16 bits. If the inputted number exceeds the limit, it cannot be store where it should be, so it is automatically wrapped by the system to become a number that can be stored in the allocated memory. (SNYK, 2025)

For example, let's say that the number 70,000 is inputted as the payment amount. The first thing that the system performs is:

$$\text{storedValue} = \text{inputValue} \bmod 65536$$
$$\text{storedValue} = 70000 \bmod 65536$$
$$70000 / 65536 = 1 \text{ remainder } 4464$$

This part is the first overflow which is the integer wraparound during the input conversion. This occurs when `(scanf("%hu", &payment))` reads in a number that is larger than 65535. As modulus is used in this calculation, the remainder of the division is the number that we set as storedValue which is 4464.

$$\text{newBalance} = 900 + \text{storedValue (payment)}$$
$$\text{newBalance} = 900 + 4464$$
$$\text{newBalance} = 5364$$

This value is then added by using `unsigned short newBalance = balance + payment;` and newBalance is checked to see if it is below the limit. If it is, the code will go through, but in this case it does not as newBalance equals 5364 which exceeds the limit.

To see it go through and have success in disrupting the amount in the account, let's take a payment of 65574. In the same way as the first,

$$\text{storedValue} = \text{inputValue} \bmod 65536$$
$$\text{storedValue} = 65574 \bmod 65536$$
$$65574 / 65536 = 1 \text{ remainder } 38$$

This value is then again added by using `unsigned short newBalance = balance + payment;` and newBalance is checked to see if it is below the limit. If it is, the code will go through. In this case the number of newBalance is below the limit of 1000 and will be accepted correctly.

$$\text{newBalance} = 900 + \text{storedValue (payment)}$$
$$\text{newBalance} = 900 + 38$$
$$\text{newBalance} = 938$$

Although accepted by the program, this isn't correct. The payment originally inputted is 65574 but when the program prints out the newBalance, is it set to read 938. This fault is due to both the wrapping at the beginning of the program and the arithmetic calculations of newBalance.

Through thorough investigation and testing of the maths behind what numbers are accepted, I have figured out a formula to calculate what ranges will result in an accepted number.

$$\text{Minimum} = 64636 * n + ((n-1) * 900)$$
$$\text{Maximum} = 65535 * n - (n-1)$$

3.5 Secure Fix and Resolution

There are a few changes that I made to the code to make it more secure. The first and more obvious change is to the order that the arithmetic operations are made in. In the vulnerable version of the code, the program first calculated the new balance and then checked whether it exceeded the stated limit. This gave the overflow opportunity to occur before it was properly checked and verified.

In the fixed version, I have rewritten it so that it performs the check first using `if (balance + payment > limit)` which will return an error once it finds that the limit has been passed. If the limit hasn't been passed, it will move onto adding the balance together using `balance = balance + payment`, then printing out a confirmation message and reprinting the new balance to state that it has worked.

It is important to validate the result of the addition before applying it to the account balance. The fixed version helps to prevent the irregularities that the wrapping causes. This helps to guarantee that the limit in the account cannot be bypassed in any obvious way, unlike in the original version.

General Prevention of Overflow Vulnerabilities

From my experience in writing purposefully vulnerable programs and learning how the overflows occur within them, the most common issue with preventing overflow vulnerabilities is the extreme lack of validation practicing within the code. C is a low-level language which in turn, puts a lot of the issue prevention on the programmer rather than the user or the system itself.

One common way of preventing overflows is by enforcing stricter bounds checking when reading in user input. In most of my examples, this can be fixed by changing functions like `scanf()` to include length restrictions in their format specifiers. By limiting the number of characters that can be stored in a buffer, the program makes sure that the input cannot reach out of the allocated memory space.

Validating numerical values is also important when working with buffers with fixed sizes. Before performing operations, it is better practice to have programs verify that the result will not exceed the maximum value that the data type can hold. Checking values helps to prevent integer wraparound and ensures that parts of the program cannot be passed through arithmetic overflows.

Thankfully, more modern languages, operating systems, and compilers include the necessary protection to help reduce any impacts of these overflow vulnerabilities. Some protections include stack protection, mechanisms, address space layout randomisation, and runtime error detection. (Lenovo, 2025)

Conclusion

This report demonstrated three common overflow-related vulnerabilities and a fix for each in C. Each example showcased poor programming practices and improper handling of user inputs which caused memory and data limits to be breached and cause unintended behaviour.

The three examples illustrate how different factors in code can result in accessing adjacent memory, overwriting the adjacent memory, or reading it back to the user. These examples highlight the importance of understanding secure programming practices while developing software in languages that may be less secure than others. By validating inputs where possible, restricting buffer sizes, and verifying arithmetic operations before they can occur, a lot can be understood about the risks of overflow vulnerabilities, how they occur, and how to better protect against them in the future.

Bibliography

- Fiveable. (2025, September). *Stack overflow Definition - Data Structures Key Term* |...
<https://fiveable.me/data-structures/key-terms/stack-overflow>
- Fortinet. (n.d.). *What Is Buffer Overflow? Attacks, Types & Vulnerabilities* | Fortinet. 2026.
Retrieved March 8, 2026, from
<https://www.fortinet.com/resources/cyberglossary/buffer-overflow>
- fprintf Documentation*. (1997).
<https://pubs.opengroup.org/onlinepubs/7908799/xsh/fprintf.html>
- him0000. (2025, November 1). *Format Specifiers in C - GeeksforGeeks*.
<https://www.geeksforgeeks.org/c/format-specifiers-in-c/>
- Kartik. (2025, October 18). *Memory Layout of C Programs - GeeksforGeeks*.
<https://www.geeksforgeeks.org/c/memory-layout-of-c-program/>
- Lenovo. (2025). *What is an Overflow Error? How to Prevent it in Programming* | Lenovo IE.
<https://www.lenovo.com/ie/msd/en/glossary/overflow-error/>
- Microsoft Learn. (2024, Spring 6). *Data Type Ranges* | Microsoft Learn.
<https://learn.microsoft.com/en-us/cpp/cpp/data-type-ranges?view=msvc-170>
- SNYK. (2025). *Integer overflow or wraparound* | Tutorial and examples | Snyk Learn.
<https://learn.snyk.io/lesson/integer-overflow-or-wraparound/?ecosystem=cpp>